

Universitat Oberta de Catalunya

Herramientas HTML y CSS II

PEC 3 - Utility-first CSS con Tailwind

Jornadas de Artesania

Alumno: Federico Javier Martino

Correo: fmartino@uoc.edu

Curso: 2025-2026

Entrega: junio de 2026

1. Introduccion y objetivos

Esta practica revisa el sitio **Jornadas de Artesania** desarrollado en la PEC 2 y lo reescribe aplicando la metodologia **utility-first CSS** vista en el modulo 4.2, usando la libreria de utilidades **Tailwind CSS v4**. No se trata de mejorar el diseno ni el contenido, sino de reescribir el HTML y el CSS de dos paginas para que sigan el paradigma de Atomic CSS, partiendo de un proyecto nuevo basado en UOC Boilerplate (empaquetador Parcel, estilos en Sass) y sin Bootstrap ni Stylelint.

Concretamente, en esta entrega:

- Se parte de un proyecto limpio basado en UOC Boilerplate y se instala y configura Tailwind v4 con el enfoque CSS-first.
- Se recrean dos paginas de la PEC 2 (**Portada** y **Ponentes**) con utilidades de Tailwind, manteniendolas responsive.
- Se extraen clases con `@apply` y componentes con `posthtml-include`.
- Se anade una tercera pagina (**Concurso**) generada con ayuda de una IA generativa a partir de un diseno de Figma.
- Se documenta el proceso y se publica el repositorio y el despliegue.

2. Entorno y proceso de desarrollo

2.1. Punto de partida y limpieza del proyecto

El proyecto parte de UOC Boilerplate (Parcel + Sass), la misma base que la PEC 2, pero como proyecto y repositorio nuevos. El primer paso fue eliminar todo lo que no debe estar en esta entrega:

- Se desinstalo **Bootstrap** y `@popperjs/core` de `package.json` y se elimino su `@import` en los estilos y el `import` de su JavaScript en `main.js`.
- Se desinstalo **Stylelint** y sus plugins, se borro `.stylelintrc` y se quito el paso de Stylelint del script de build, que queda como `npm-run-all clean parcel:build`.
- Se mantuvieron `.npmrc` (con `legacy-peer-deps=true`) y `.posthtmlrc` (posthtml-include).

2.2. Instalacion de dependencias

Sobre el proyecto limpio se instalaron Tailwind v4 y su plugin de PostCSS, ademas de las fuentes e iconos:

```
npm install tailwindcss @tailwindcss/postcss --save-dev
npm install @fontsource/playfair-display @fontsource/source-sans-pro @fortawesome/fontawesome
```

Tras editar `package.json` se ejecuto `npm install`, que resolvió el árbol de dependencias sin Bootstrap ni Stylelint.

2.3. Configuración de Tailwind v4 (CSS-first)

Tailwind v4 abandona el fichero `tailwind.config.js` de la v3 y se configura directamente desde el CSS. La integración con Parcel + Sass se hizo en tres puntos:

a) Plugin de PostCSS. Se creo `.postcssrc` en la raíz:

```
{
  "plugins": {
    "@tailwindcss/postcss": {}
  }
}
```

b) Importación de Tailwind. En `_dependencies.scss` se importa Tailwind con extensión `.css` para que Dart Sass lo deje pasar como import de CSS plano y sea `@tailwindcss/postcss` quien lo resuelva. Las fuentes y los iconos se importan sin el prefijo `npm:`:

```
@import "tailwindcss/index.css";
@import "@fortawesome/fontawesome-free/css/all.css";
@import "@fontsource/playfair-display/400.css";
/* ... resto de pesos ... */
```

c) Tema. En `_variables.scss` se define el tema con `@theme`, reutilizando la paleta, las tipografías, los breakpoints, los radios y las sombras de la PEC 2 como tokens de Tailwind:

```
@theme {
  --font-sans: "Source Sans Pro", "Segoe UI", Arial, sans-serif;
  --font-serif: "Playfair Display", Georgia, serif;
  --text-display: 2rem;
  --text-headline: 2.75rem;
  --text-hero: 4rem;
  --color-clay: #b85c2c;
  --color-cream: #faf3e6;
  --color-ink: #2b2018;
  /* ... paleta completa, breakpoints sm/md/lg/xl, radios y sombras ... */
}
```

Así, utilidades como `text-clay`, `bg-cream`, `font-serif` o `text-hero` generan exactamente los valores del tema. El orden de carga en `main.scss` es: variables (tema) -> dependencias (Tailwind, fuentes, iconos) -> base -> componentes.

2.4. Entorno de desarrollo

Durante el desarrollo se usa `npm run dev`, que limpia la salida y levanta Parcel en `http://localhost:8123` con recarga en caliente. Parcel procesa cada `.scss` con la cadena Sass -> PostCSS (Tailwind) -> CSS, y resuelve los `<include>` de posthtml-include.

2.5. Compilación para producción

La compilación se hace con `npm run build` (`clean` + `parcel:build`), que deja en `dist/` el HTML, el CSS minificado, el JS y los recursos (imágenes y fuentes con hash). Tailwind solo emite las utilidades realmente usadas y, por su *tree-shaking*, solo las variables de tema referenciadas, por lo que la hoja final es pequeña.

2.6. Publicación

El código se publica en un repositorio de GitHub nuevo y se despliega en Netlify (build `npm run build`, publish `dist`), distintos de los de la PEC 2. Las URLs figuran en el apartado 7.

3. Construcción de las paginas

3.1. Decisiones generales

Se eligieron **Portada** e **Ponentes** por ser las paginas con mas estructuras repetidas (tarjetas de informacion, destacados, fichas de ponente y actividades), ideales para demostrar la extraccion de clases y componentes. La portada se mantiene como `index.html`.

La arquitectura de estilos quedo minima y separada en capas de Tailwind:

- `_variables.scss`: tema `@theme` (y la paleta heredada como variables Sass de apoyo).
- `modules/_base.scss`: capa `base` con la tipografia global, el color de enlaces y el enlace de salto accesible.
- `modules/_components.scss`: capa `components` con las clases extraidas via `@apply`.

Las hojas de estilo por pagina de la PEC 2 (cientos de lineas de SCSS con BEM) desaparecen: el estilo vive ahora en las clases utilitarias del HTML.

3.2. Portada (index.html)

- **Hero / cartel:** el grid con `grid-template-areas` de la PEC 2 se rehizo con un grid responsive (`md:grid-cols-[1.3fr_1fr]`): en movil se apila el texto y la imagen; en escritorio, texto a la izquierda e imagen vertical a la derecha. El fondo (gradiente de marca multicapa) se mantiene como clase `.hero-bg` por ser difícil de expresar solo con utilidades.
- **Banda de informacion:** grid de 1 a 3 columnas con el componente `info-item`.
- **Destacados:** grid de 1/2/3 columnas con el componente `highlight-card`, que conserva la **container query** de la PEC 2 (la tarjeta pasa a horizontal cuando su propio ancho supera 480px) mediante las utilidades `@container` y `@[480px]:`.
- **Galeria:** el carrusel de Bootstrap, que requeria JavaScript, se representa de forma estatica (primera imagen, leyenda, indicadores y flechas visuales), tal y como permite el enunciado.

3.3. Ponentes (ponentes.html)

- **Cabecera interior:** banner con gradiente (`.banner-bg`), titulo, subtítulo y miga de pan.
- **Filtros por disciplina:** botones con la clase `.btn-filter`. El filtrado lo sigue gestionando el JavaScript propio (`main.js`), que es vanilla y no dependia de Bootstrap; solo alterna la clase `.active` y muestra u oculta las fichas.
- **Fichas de ponente:** ocho instancias del componente `speaker-card`, incluida la variante destacada (keynote) a ancho completo. Conservan la container query (a partir de 520px pasan a horizontal).
- **Programa por dia:** el acordeon de Bootstrap se sustituyo por el elemento nativo `<details><summary>`, que es plegable sin JavaScript; la flecha gira con la variante `group-open` de Tailwind.

3.4. Cabecera y pie

Son componentes compartidos (`header.html` y `footer.html`) reescritos con utilidades. La navbar de Bootstrap se sustituye por un `<nav>` flex: en movil se oculta el menu de escritorio (`hidden lg:flex`) y se muestra el icono de hamburguesa (`lg:hidden`), sin desplegar el menu al pulsar, como permite el enunciado. El enlace activo se resalta con `aria-[current=page]:text-clay-dark`.

3.5. Problematicas surgidas y soluciones

Problema	Solucion
Dart Sass falla al importar <code>tailwindcss</code> (lo busca como parcial Sass).	Importarlo como <code>"tailwindcss/index.css"</code> : la extension <code>.css</code> hace que Sass lo deje pasar y Tailwind lo resuelva en PostCSS.
Resolucion de FontAwesome y Fontsource con Tailwind en la cadena.	Importarlos sin el prefijo <code>npm:</code> . El build resuelve y reescribe correctamente las rutas de las fuentes (<code>url()</code>) en <code>dist/</code> .
Usar <code>@apply</code> dentro de ficheros <code>.scss</code> .	Funciona porque todo compila a una unica hoja que incluye el import de Tailwind. Se evitan variantes con dos puntos dentro de <code>@apply</code> (se aplican en el HTML o con selectores <code>:hover</code> anidados).
Componentes de Bootstrap con JS (carrusel, acordeon).	Carrusel: version estatica. Acordeon: elemento nativo <code><details></code> , que aporta funcionalidad real sin JS.
Tarjetas repetidas con contenido distinto.	posthtml-include con <code>locals</code> (interpola con <code>{{ }}</code>), condicionales <code><if></code> y bucles <code><each></code> , que trae como dependencia posthtml-expressions.

4. Clases y componentes extraídos

La documentación de Tailwind recomienda, ante la duplicación, usar plantillas/partials para los bloques complejos y reservar las clases con `@apply` para patrones pequeños que se repiten mucho. Se han aplicado ambas estrategias.

4.1. Clases con `@apply` (`modules/_components.scss`)

Clase	Por que se extrae
<code>.btn</code> + <code>.btn-clay</code> + <code>.btn-cream-outline</code>	El mismo conjunto de utilidades de botón aparece en la cabecera y en el hero. La base y las variantes evitan repetir 8-10 utilidades en cada botón.
<code>.btn-filter</code>	Los cinco filtros de ponentes comparten estilo; además su estado <code>.active</code> (que activa el JS) se define una sola vez.
<code>.pill</code>	Badge/etiqueta redondeada usada en el hero, los destacados y las fichas; el color se pasa por utilidad para reutilizar la forma.
<code>.card</code>	Tarjeta blanca con borde y esquinas redondeadas común a destacados, ponentes y actividades.
<code>.section-heading</code>	Titular de sección en tipografía serif, repetido en cada bloque.

También se mantienen como CSS (no `@apply`) los fondos decorativos `.hero-bg` y `.banner-bg`, por ser gradientes multicapa poco prácticos de escribir con utilidades.

4.2. Componentes con `posthtml-include` (`src/views/`)

Componente	Uso
<code>info-item.html</code>	Tarjeta de información práctica (3 usos en la portada).
<code>highlight-card.html</code>	Tarjeta destacada con container query (6 usos en la portada).
<code>speaker-card.html</code>	Ficha de ponente parametrizada (8 usos en ponentes), con bucle <code><each></code> para los enlaces.
<code>activity.html</code>	Actividad del programa (9 usos en ponentes).

Cada componente recibe sus datos con el atributo `locals`. Así, el marcado de una ficha de ponente se escribe una sola vez y se reutiliza con distintos datos, eliminando la duplicación sin renunciar al `utility-first`. La cabecera y el pie también son includes, pero no se cuentan entre estos tres mínimos.

5. Preguntas sobre utility-first CSS

5.1. CSS semantico frente a CSS de utilidades

En las PEC anteriores se uso **CSS semantico** con BEM: clases con significado (`.speaker__name` , `.home-highlight__tag`) y el estilo en hojas SCSS separadas. Con el **CSS de utilidades** el estilo se compone directamente en el HTML con clases atómicas (`flex` , `gap-2` , `text-clay`).

Como afecta al proceso de desarrollo:

- Se deja de saltar entre el HTML y el SCSS: se maquetaba y se estiliza en el mismo sitio, lo que acelera mucho el trabajo y el ensayo-error.
- Desaparece el esfuerzo de *nombrar* las clases (uno de los problemas difíciles del CSS), y con ellas las dudas de la nomenclatura BEM.
- El tema (`@theme`) centraliza las decisiones de diseño, así que los colores y tamaños siguen siendo coherentes pese a escribirse en el HTML.
- El control de la especificidad casi desaparece: las utilidades tienen especificidad plana y el orden lo gestiona la capa `utilities` .

Como afecta al código:

- El SCSS adelgaza de forma drástica: cientos de líneas de estilos por página se sustituyen por unas pocas clases `@apply` y la capa base.
- El HTML, en cambio, engorda: las listas de clases son largas. La duplicación se controla con componentes (posthtml-include) y con `@apply` .
- El CSS final es más pequeño y predecible (solo se generan las utilidades usadas), frente al CSS semántico, que tiende a crecer y a acumular reglas muertas.

5.2. Librería de componentes frente a librería de utilidades

Bootstrap es una **librería de componentes**: ofrece piezas ya diseñadas (navbar, card, carousel, accordion) con su propio aspecto. Tailwind es una **librería de utilidades**: no trae componentes, sino piezas mínimas con las que se construye cualquier diseño.

	Componentes (Bootstrap)	Utilidades (Tailwind)
Arranque	Muy rapido: se pega el componente y funciona.	Mas lento al principio: hay que construir cada pieza.
Personalizacion	Cuesta salirse del estilo de la libreria; a menudo hay que sobrescribir.	Total: el diseno es propio desde el primer momento.
JavaScript	Componentes interactivos (carrusel, acordeon) dependen de su JS.	No incluye JS; la interaccion se resuelve aparte o con HTML nativo.
Aspecto	Tiende a un "aire Bootstrap" reconocible.	Sin aspecto por defecto: cada sitio es distinto.
Peso final	Suele incluir CSS que no se usa.	Solo se genera lo utilizado.

En la practica se noto en los componentes con JS: el carrusel y el acordeon de Bootstrap "venian gratis", pero con Tailwind hubo que decidir como resolverlos (carrusel estatico y acordeon con `<details>`). A cambio, el resultado es a medida y sin dependencias de JavaScript de terceros.

6. Pagina generada con una IA generativa

6.1. Objetivo y diseno de partida

El enunciado pide usar una IA generativa para crear el HTML y el CSS de una pagina a partir de un diseno de Figma, afinando los prompts para acercarse lo maximo posible al diseno y analizando el resultado. La pagina resultante (`concurso.html`) es autocontenida (HTML con su propio CSS, sin Tailwind, como permite el enunciado) y replica el diseno adaptando el contenido a la tematica artesana (un "Concurso de Artesania 2026").

6.2. Secuencia de prompts

Prompt 1. "A partir de esta imagen de Figma, genera una pagina HTML con su CSS (sin frameworks) que replique el diseno: una barra superior con marca a la izquierda y un menu con un boton oscuro a la derecha; un titular centrado con subtítulo; una imagen ancha; una seccion con un titulo y tres tarjetas, cada una con titulo, parrafo y un autor con avatar; y un pie con el nombre, iconos sociales y tres columnas de enlaces. Usa un CSS limpio y un contenedor centrado de unos 1080px."

Prompt 2. "Adapta el contenido a la tematica de unas jornadas de artesanía: titulo 'Participantes', tres piezas (Cantaro vidriado, Cesto de mimbre, Bandeja de nogal) con autores artesanos, y el pie con columnas Piezas / Participantes / Ediciones (2026, 2025, 2024). Manten el aspecto neutro y claro del diseno original."

Prompt 3. "Hazla responsive: las tres tarjetas a una columna en moviles y el pie a dos y luego una columna. Reduce el titular en pantallas pequenas."

Prompt 4. "Los iconos sociales se ven como cuadrados rotos porque usabas clases de FontAwesome y la pagina no carga esa libreria. Sustituyelos por SVG en linea (facebook, linkedin, instagram, youtube) que hereden el color del texto."

6.3. Errores cometidos por la IA y correcciones

- **Iconos inexistentes:** la primera version uso clases `fa-*` de FontAwesome sin cargar la libreria, asi que los iconos sociales no aparecian. Se corrigio pidiendo SVG en linea (prompt 4).
- **Proporciones y espaciados:** el primer resultado tenia menos aire que el diseno (titular pegado a la imagen, tarjetas muy juntas). Se ajustaron margenes y `padding` para acercarlo al Figma.
- **Tipografia:** la IA puso todo en una sans-serif; en el diseno el titular es serif. Se cambio el titular a una serif (Georgia) para parecerse mas.
- **Contenido generico:** mantuvo textos del diseno original ("recipes", "contestants") hasta que se le pidio explicitamente adaptarlo a la tematica artesana.

6.4. Corregir frente a escribir de cero. Opinion

Partir del código de la IA fue más rápido que escribir la página desde cero: en pocos minutos había una estructura completa y razonable que solo necesitaba ajustes. Las correcciones fueron sencillas porque el código generado era limpio y predecible (CSS plano, nombres de clase claros); los problemas eran puntuales (iconos, espaciados, tipografía) y fáciles de localizar.

La dificultad de corregir crece cuando el código generado es enrevesado o usa técnicas que uno no domina; en ese caso a veces compensa rehacerlo. Aquí no fue el caso. **Mi opinion** es que estas herramientas son muy útiles como punto de partida y para tareas repetitivas o de andamiaje, siempre que se entienda el código que producen: hay que revisarlo, porque cometen errores (referencias a librerías no cargadas, detalles de accesibilidad, fidelidad al diseño) y tienden a soluciones genéricas. Son un acelerador, no un sustituto del criterio propio.

7. Enlaces

Recurso	URL
Repositorio GitHub (PEC 3)	<pendiente de publicar: nuevo repositorio>
Despliegue Netlify (PEC 3)	<pendiente de publicar: nuevo deploy>
Despliegue PEC 2 (referencia)	https://jornadas-artesania-uoc.netlify.app/
Video explicativo	<pendiente: enlace de Google Drive (@uoc.edu)>

8. Conclusiones

Reescribir el sitio con utility-first cambia por completo el flujo de trabajo: se maquetaba y se estilaba a la vez, el CSS propio casi desaparece y la coherencia se mantiene gracias al tema. El reto se desplaza a controlar la duplicación en el HTML, que se resuelve bien con clases `@apply` y con componentes de `posthtml-include`. Frente a una librería de componentes como Bootstrap, Tailwind exige más trabajo inicial pero da un control total del diseño y un CSS final más ligero. La experiencia con la IA generativa confirma su utilidad como punto de partida, siempre con revisión crítica del código.